THE OPEN SOURCE AUDIT

A Capstone Project — Open Source Software (OSS NGMC)

Chosen Software: Git

Student Name: Vaibhav

Registration Number: 24BCE10717

Date: 30th March 2026

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem Git Was Created to Solve

In the early 2000s, the Linux kernel project — one of the most active open-source development efforts in history — faced a surprisingly unglamorous problem: version control. Thousands of developers worldwide were contributing patches to a codebase that was growing at an extraordinary rate, and the team had no adequate tool to manage this.

For a time, the kernel project relied on a proprietary distributed version control system called BitKeeper, made by a company called BitMover. BitMover allowed the kernel team to use it free of charge under a restrictive agreement: no one working on a competing open-source version control tool could use BitKeeper. It was a fragile peace — a community built on the principle of software freedom, dependent on a tool it did not own, could not read, and could not modify.

The arrangement collapsed in April 2005 when Andrew Tridgell — a developer famous for creating Samba — began reverse-engineering the BitKeeper protocol. BitMover revoked the free licence. Linus Torvalds, the creator of Linux, was left without a version control system for one of the world's most important software projects.

Rather than adopt another existing tool — CVS was too slow and too centralised; Subversion was not much better — Torvalds decided to write his own. His requirements were stark and non-negotiable: it had to be fast (he quoted CVS's speed as an example of what not to do), it had to support distributed workflows (so every developer had a full copy of history), and it had to guarantee data integrity through cryptographic hashing.

Within ten days of beginning development in April 2005, Torvalds had a working prototype capable of hosting its own development. He named it Git — a British slang term for an unpleasant person. Torvalds has said the name was self-deprecating: he names projects after himself. The early README file was famously blunt: 'git' is an arbitrary three-letter combination that is pronounceable and not used by any common UNIX command.

The deeper significance of Git's creation is not merely technical. It was a direct consequence of what happens when a free software community depends on proprietary infrastructure. The moment BitMover withdrew its licence, the Linux kernel project was exposed. Git was built as a solution to that vulnerability — and in solving it, it accidentally created the foundation for all modern software collaboration.

Today, Git is estimated to be used by over 90% of professional developers worldwide. GitHub, GitLab, Bitbucket, and countless other services are built on top of it. The irony is that a tool built in a week of frustrated urgency by one person became the universal infrastructure for collaborative software development for the next two decades.

A2. The Licence — What It Actually Says

Git's Licence: GPL v2

Git is distributed under the GNU General Public License version 2 (GPL v2), the same licence that governs the Linux kernel. This is not a coincidence — Torvalds chose it because it matched his philosophy and because it was already the licence he was most familiar with.

The Four Freedoms of Free Software

Richard Stallman's GNU Project defines free software through four essential freedoms. A licence either grants all four or it is not truly free:

- Freedom 0: The freedom to run the program for any purpose, without restriction.
- Freedom 1: The freedom to study how the program works and to change it. This requires access to the source code.
- Freedom 2: The freedom to redistribute copies so you can help others.
- Freedom 3: The freedom to distribute your modified versions to others. This allows improvements to benefit the whole community.

Git's GPL v2 licence grants all four freedoms. You may download Git, run it, read every line of its C source code, modify it to behave differently, and distribute your modified version — provided that you also distribute it under the GPL v2 and make the source available to whoever receives your binary.

Can a Company Modify and Sell Git Without Sharing Changes?

No. This is the GPL's most important and most contested clause: the copyleft requirement. If a company takes Git's source code, modifies it, and distributes the resulting binary to users outside their organisation, they must release the modified source code under the GPL v2. They cannot make their changes proprietary.

This is why Git cannot become what happened to Unix. Unix's source code was once freely shared among universities; its commercialisation and fragmentation across proprietary variants is precisely the kind of outcome the GPL was designed to prevent. A company may sell a product that includes Git, but the GPL travels with the code.

Note that this applies to distribution. A company can modify Git internally and never release it, and they have no obligation to share. The GPL governs distribution, not use.

GPL vs MIT: A Philosophical Divide

The GPL and the MIT licence represent two different philosophies within the open-source world. The MIT licence is permissive: it grants all four freedoms but imposes no requirement on downstream users. A company may take MIT-licensed code, incorporate it into a proprietary product, never release the source, and pay nothing. The original author gets credit (the licence requires attribution) but no guarantee that their work remains open.

The GPL is reciprocal: it grants the same freedoms but requires that they be passed on. If you build on GPL code and distribute the result, you must distribute it under the GPL. The freedom is self-perpetuating, which is why Stallman called this 'copyleft' — it uses copyright law to enforce openness rather than restrict it.

For something I built with the intention of it remaining permanently open — infrastructure, a development tool, anything I wanted to remain a public good — I would choose the GPL. For a library where I wanted maximum adoption and was willing to accept proprietary use in exchange for reach, I might choose MIT. The choice is not just technical; it reflects what you believe about software and community.

Free as in Free Beer vs Free as in Freedom

The distinction is one of the most important in open-source philosophy. 'Free as in free beer' means no monetary cost — you can acquire and use the software without paying. 'Free as in freedom' (or 'libre') means you have the rights to use, study, modify, and redistribute the software.

These two meanings frequently overlap but do not always coincide. Git is both: it costs nothing and you have full rights over it. But consider a proprietary tool offered without charge — a free-to-use SaaS product with a closed codebase. That is free as in free beer but not free as in freedom. You cannot see how it works, cannot fix its bugs, cannot modify it to fit your needs, and cannot continue using it if the company shuts down or changes its terms.

For Git specifically: its GPL v2 licence ensures that freedom does not evaporate the moment a company gets involved. The licence is the legal instrument that makes 'free as in freedom' meaningful rather than aspirational.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The argument for universal open source is compelling on principle: software increasingly mediates nearly every aspect of human life — healthcare systems, financial infrastructure, communication, education. When the code governing these systems is closed, it is unauditable, unaccountable, and potentially dangerous. Citizens and users of these systems have no way to verify that their interests are being served rather than exploited. Open source is, in this sense, a form of democratic accountability applied to technology.

The argument against is pragmatic. Software development is expensive. It requires sustained effort, infrastructure, and skilled people who need to be paid. Many categories of software — games, specialist professional tools, proprietary algorithms that constitute a company's core competitive advantage — would simply not be built at all if there were no way to generate revenue from them. Open source thrives in infrastructure, tooling, and ecosystems; it has historically struggled to fund polished, consumer-facing products without a business model.

A reasonable middle ground is that software which runs critical public infrastructure — operating systems, healthcare systems, voting machines, financial platforms — should be open to audit even if not freely redistributable. The distinction between 'open to inspection' and 'open to modification and redistribution' may be the more productive debate.

The Ethics of Building Profits on Community Labour

This is one of the most active ethical debates in modern open source. Companies like Red Hat, Google, and Microsoft contribute enormously to Linux — but they also derive enormous value from it. Red Hat was acquired by IBM for 34 billion US dollars; its entire product is essentially Linux with enterprise support. Is that extraction, or is it a valid business model that also sustains the ecosystem?

The clearest ethical problem arises not with companies that contribute back — Google employs kernel developers; Microsoft is now one of the largest contributors to the Linux kernel — but with those that consume without giving. The term 'open-source sustainability' describes the crisis where foundational libraries are maintained by one or two volunteers working without compensation while trillion-dollar companies ship them in commercial products. The Heartbleed vulnerability in OpenSSL — a library that secured trillions of dollars of internet transactions, maintained by a handful of unpaid people — was a defining moment for this conversation.

The GPL is one answer to this: by requiring that improvements to GPL software be returned to the community, it structurally prevents the pure-extraction model. Git, licensed under GPL v2, cannot be forked into a closed proprietary tool without its source code being released.

Standing on the Shoulders of Giants

The phrase, attributed to Isaac Newton, describes how each generation of scientists builds on the documented knowledge of those before them. In software, this is literal and traceable. Every Python script runs on an interpreter built in C. Every website is served by Apache or Nginx, sitting on a Linux kernel. Every Git push travels through OpenSSL. The entire modern internet is, at every layer, built on open-source foundations that were shared freely by their authors.

Open source does not undermine original innovation — it accelerates it. By removing the need to rebuild foundational components from scratch, it allows developers to work at increasingly higher levels of abstraction. The question of whether it enables or undermines innovation is answered empirically: the most innovative software companies in the world — Google, Amazon, Meta, Stripe — are built almost entirely on open-source infrastructure. They exist because the foundations were free.

Part B — Linux Footprint (Unit 2)

Installation Method

Git is available in the default package repositories of virtually every major Linux distribution. It does not require compilation from source for standard use, making it one of the most accessible open-source tools on the platform.

- Debian/Ubuntu (APT): sudo apt update && sudo apt install git
- Fedora/RHEL/CentOS (DNF/YUM): sudo dnf install git
- Arch Linux (Pacman): sudo pacman -S git
- From source: download tarball from git-scm.com, run ./configure && make && sudo make install

The package manager installation is preferred because it integrates Git with the system's update mechanism, ensuring security patches are applied automatically when the distribution releases them.

Key Directories

After installation, Git's files are distributed across standard Linux filesystem hierarchy locations:

- /usr/bin/git — the main Git executable binary
- /usr/lib/git-core/ — Git's supporting sub-commands and helper binaries (e.g. git-fetch, git-merge)
- /usr/share/man/man1/ — manual pages for git commands
- /usr/share/doc/git/ — documentation and licence files
- /etc/gitconfig — system-wide configuration (applies to all users on the machine)
- ~/.gitconfig or ~/.config/git/config — per-user configuration (name, email, aliases, preferred editor)
- .git/ — per-repository directory created by git init, containing objects, refs, HEAD, config, and hooks

User and Group

Git runs as the invoking user. There is no dedicated git system user for local repositories — this is by design. When you run git commit, Git executes as you, writes objects to the .git/ directory you own, and has no elevated privileges.

This matters for security: Git's design prevents a compromised repository from escalating privileges. Contrast this with a service like a web server (Apache runs as www-data) or a database (MySQL runs as mysql), which need a dedicated unprivileged user to limit damage if they are exploited. Git's interactive, per-user nature means the security boundary is your Linux user account.

For shared repositories — such as those on a server — a dedicated git system user (as used by GitLab, Gitea, and similar tools) is created, and repositories are owned by that user with group permissions controlling who can read and write.

Service Management

Git itself is not a daemon and does not run as a system service for normal local use. However, it can run as a service in two modes:

- git daemon — a lightweight, unauthenticated read-only server for serving public repositories over the git:// protocol. Controlled with: sudo systemctl start git-daemon (if packaged as a service unit)
- SSH-based access — most production Git hosting uses SSH, where the sshd service (not Git itself) is the persistent daemon. Git is invoked per-connection.

For local development, no service management is needed. Git is simply invoked as a command-line tool.

Update Model

Security patches for Git reach your machine through your Linux distribution's package management system. When the Git security team (coordinated through the Git mailing list and the oss-security mailing list) discovers or receives a vulnerability report, they prepare a patch, coordinate a disclosure date, and release a new version. Your distribution's package maintainers then build the updated package and push it to the official repository.

Running sudo apt upgrade (Debian/Ubuntu) or sudo dnf upgrade (Fedora/RHEL) applies any pending security patches including Git updates. This is the open-source security model in practice: transparent disclosure, community review of the patch, and distribution through a trusted package infrastructure.

Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies

Git is written primarily in C and Shell, with minimal external dependencies by design — Torvalds explicitly wanted it to be portable and self-contained. Key dependencies include:

- OpenSSL or libgcrypt — for SHA-1/SHA-256 cryptographic hashing (every Git object is identified by its hash)
- zlib — for compressing stored objects (Git pack files use zlib deflate compression)
- libcurl — for HTTPS transport (git clone https://...)
- POSIX shell — many Git sub-commands and hooks are shell scripts
- Perl — used for some bundled tools like git-svn and git-send-email

The deliberate minimalism of Git's dependencies is itself a design philosophy: a tool that controls the history of your entire codebase should not have a fragile dependency chain that could break at any moment.

What Git Has Enabled

Git's impact on the software ecosystem is difficult to overstate. It has directly enabled or inspired:

- GitHub (2008) — built entirely on Git, GitHub became the world's largest software collaboration platform, hosting over 100 million repositories. Acquired by Microsoft in 2018 for 7.5 billion USD.
- GitLab (2011) — an open-source Git platform that can be self-hosted, providing CI/CD, issue tracking, and container registry alongside version control
- Gitea / Forgejo — lightweight self-hosted Git services for individuals and organisations who want sovereignty over their code infrastructure
- GitHub Actions / GitLab CI — CI/CD systems built around Git events (push, pull request, tag), making Git the trigger for automated testing and deployment pipelines
- GitOps — an operational pattern in which infrastructure state is managed through Git commits, making version control the single source of truth for both code and infrastructure
- Gerrit — Google's code review system, built on Git, used internally and by Android development

Git and the Web / LAMP Stack

Git is not part of the classic LAMP stack (Linux, Apache, MySQL, PHP) but it sits alongside it as the universal delivery mechanism. Every LAMP application today is developed using Git, versioned in Git, and deployed via Git. The typical modern web deployment pipeline is: git push → CI/CD pipeline triggered → automated tests run → if passing, deploy to server running Apache/Nginx on Linux with MySQL/PostgreSQL.

Git transformed software delivery from the manual copying of files to a disciplined, auditable, rollback-capable process. The concept of infrastructure-as-code — managing server configuration in Git repositories — means that Git now governs not just applications but the servers those applications run on.

Community and Governance

Git's community is one of the most technically rigorous in open source. Development happens primarily through the git@vger.kernel.org mailing list — one of the oldest forms of internet communication still in active use for a major project. There is no GitHub pull request workflow for Git itself; patches are submitted as formatted emails, reviewed in public threads, and applied by the maintainer.

The current maintainer is Junio C Hamano (also known as gitster), who has held this role since Torvalds stepped back in 2005. Hamano coordinates a team of trusted sub-system maintainers and a larger community of occasional contributors. Major corporate contributors include Google, Microsoft, GitHub, Atlassian, and Alibaba — all companies with significant commercial interests in Git's continued health.

The project holds an annual Git Merge conference, and community discussions happen through the mailing list, IRC channels (#git on Libera.Chat), and the Git Gitgadget tool which bridges GitHub PRs to the mailing list for contributors who prefer the pull-request workflow.

Part D — Open Source vs Proprietary (Critical Analysis)

Comparison: Git vs Proprietary Alternatives

The closest proprietary alternatives to Git are Perforce Helix Core (used in game development and large binary projects), Microsoft's Team Foundation Version Control (TFVC, now largely superseded by Azure Repos with Git), and BitKeeper — the very tool whose withdrawal triggered Git's creation.

Dimension	Git (Open Source)	Proprietary Alternative
Cost	Free to download, use, modify, and redistribute under GPL v2. No licensing fees ever.	GitHub (owned by Microsoft) is free for public repos but charges for private enterprise features. GitLab (SaaS) has paid tiers.
Security (code audit)	Full source code is public. Any researcher worldwide can audit git's C code for vulnerabilities. Patches are community-reviewed.	Proprietary tools like Perforce or Azure DevOps have closed source. Security depends entirely on the vendor's internal audit.
Support & Reliability	Community support via mailing lists, Stack Overflow, and extensive documentation. No SLA, but widely supported by distros.	Commercial vendors offer paid SLAs and enterprise support. Useful for organisations needing guaranteed response times.
Freedom to Modify	Complete freedom. You can fork git, patch it, and redistribute. The GPL ensures any distribution shares source code.	Impossible. Binary-only distribution. You cannot change behaviour or fix bugs yourself without vendor cooperation.
Community vs Corporate	Maintained by a global volunteer+company coalition (Junio Hamano leads; Google, Microsoft, GitHub contribute). No single owner.	Controlled by one vendor. Roadmap, pricing, and feature decisions are made by corporate strategy, not user need.
Overall Verdict	Git is the clear choice for any development workflow, education, and open collaboration. Near-universal industry adoption.	Proprietary VCS tools are rarely used for new projects today. They persist in legacy enterprise environments.

Conclusion and Recommendation

Git is, without qualification, the right choice for any software project today — from a single student's coursework repository to a multinational engineering team coordinating across time zones. Its distributed architecture means no single server is a point of failure; every clone is a full backup. Its cryptographic integrity means history cannot be silently rewritten. Its GPL v2 licence means it will remain open regardless of what any single company decides to do with it.

The more interesting question is whether I would contribute to it. The Git mailing list is technically demanding — contributors are expected to produce clean, well-tested patches and to engage with detailed, sometimes blunt code review. But it is also one of the most consequential

codebases a developer can contribute to. Every improvement to Git's performance, correctness, or usability is immediately felt by tens of millions of developers. Having studied its origin, its philosophy, and its architecture as part of this audit, contributing even a small documentation fix or a targeted bug report feels like a meaningful act of participation in something that genuinely matters.

Part E — Shell Script Documentation (Unit 1, 2 & 5)

Five shell scripts were written and tested on Linux. Each is submitted as a separate .sh file. Documentation for each script is presented below, including its purpose, shell concepts demonstrated, and a description of expected output.

Script 1 — System Identity Report

Script	script1_system_identity.sh
Purpose	Variables (KERNEL, USER_NAME, UPTIME); command substitution with $() to capture output of uname -r, whoami, uptime -p, date; echo with formatted output strings; cat /etc/os-release with grep and cut to extract distribution name; printf for aligned columns.
Concepts	
Code (see .sh file)	See submitted script file. All scripts include inline comments on every non-trivial line.

Script 2 — FOSS Package Inspector

Script	script2_package_inspector.sh
Purpose	if-then-else for package detection; command -v to test tool availability; rpm -qi and dpkg -l for package information; grep -E for pattern matching on multiple fields; case statement with multiple patterns (git, httpd, mysql, vlc, firefox, python, libreoffice); pipe chaining.
Concepts	
Code (see .sh file)	See submitted script file. All scripts include inline comments on every non-trivial line.

Script 3 — Disk and Permission Auditor

Script	script3_disk_permission_auditor.sh
Purpose	Array declaration and iteration with for DIR in '${DIRS[@]}'; conditional [-d '$DIR'] to test directory existence; ls -ld piped to awk for permission/owner/group extraction; du -sh for human-readable size; printf for formatted tabular output; 2>/dev/null to suppress permission errors.
Concepts	
Code (see .sh file)	See submitted script file. All scripts include inline comments on every non-trivial line.

Script 4 — Log File Analyzer

Script	script4_log_analyzer.sh
Purpose	Command-line arguments $1 and $2 with default value assignment (KEYWORD=${2:-'error'}); while IFS= read -r loop for safe line-by-line file reading; if-then for keyword matching with grep -iq; counter variable with $((COUNT + 1)); bash array with += for accumulating matches; array slicing for last-5 display; do-while simulation using a boolean flag; exit codes 0 and 1.
Concepts	
Code (see .sh file)	See submitted script file. All scripts include inline comments on every non-trivial line.

Script 5 — Open Source Manifesto Generator

Script	script5_manifesto_generator.sh
Purpose	Interactive input with read -p; string interpolation inside double-quoted echo strings; file writing with > (create/overwrite) and >> (append); date command for timestamp; whoami for username-based filename; input validation with [-z] to check for empty strings; function definition as alias equivalent; cat to display the final file.
Concepts	
Code (see .sh file)	See submitted script file. All scripts include inline comments on every non-trivial line.

References

All sources below were consulted for original research and understanding. No text has been copied verbatim.

- Torvalds, L. (2005). Git initial commit message and README. kernel.org/pub/software/scm/git/
- Chacon, S. and Straub, B. (2014). Pro Git, 2nd Edition. Apress. Available freely at: git-scm.com/book
- GNU Project. The Free Software Definition. gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition. opensource.org/osd
- Raymond, E.S. (1999). The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- SPDX License List — GPL v2 full text. spdx.org/licenses/GPL-2.0-only.html
- Git project mailing list archives. lore.kernel.org/git/
- Hamano, J.C. Git maintainer notes and release announcements. github.com/gitster
- The Linux Command Line, Shotts, W. linuxcommand.org
- GNU Bash Manual. gnu.org/software/bash/manual